

# Errors, Failures, and Risks (ত্রুটি, ব্যর্থতা এবং ঝুঁকি)

## Trusting Computers

- What can go wrong?
- Case Studies
- Increasing Reliability and Safety
- Computer Models
- Perspectives on Failures, Dependence, Risk and Progress
- Learning from other technologies

## ১. Trusting Computers — মূল ধারণা

আমরা দৈনন্দিন জীবনে কম্পিউটার সিস্টেমের উপর অনেক বেশি নির্ভরশীল (dependent) — ব্যাংকিং, চিকিৎসা, বিমান চলাচল, প্রতিরক্ষা — সব জায়গাতেই। কিন্তু প্রশ্ন হলো, এই সিস্টেমগুলোর উপর আমরা কতটা বিশ্বাস (trust) করতে পারি?

এই টপিকে আমরা দেখব:

- কম্পিউটার সিস্টেম কেন এবং কীভাবে ব্যর্থ (fail) হয়
  - Real-world case studies (Therac-25, Patriot Missile)
  - কীভাবে reliability (নির্ভরযোগ্যতা) এবং safety বাড়ানো যায়
  - Computer models কতটা বিশ্বাসযোগ্য
  - Risk নিয়ে বিভিন্ন দৃষ্টিভঙ্গি (perspectives)
-

## What can go wrong?

- Facts about computer errors
  - Error-free software is very, very difficult to achieve.
  - Errors are often caused by several factors.
  - Errors can be reduced by following good software development procedures.
  - Confidence in code can be obtained by keeping it small enough to be understood.

## ২. What Can Go Wrong? — কী কী ভুল হতে পারে

### ২.১ Computer Errors সম্পর্কে গুরুত্বপূর্ণ Facts

1. **Error-free software** পাওয়া প্রায় অসম্ভব — যত বড় সিস্টেম, তত বেশি সম্ভাবনা bug থাকার। কোনো সফটওয়্যারই ১০০% bug-free গ্যারান্টি দেওয়া যায় না।
2. **Errors** সাধারণত একাধিক কারণে হয় (**multiple factors**) — একটামাত্র কারণে বড় failure খুব কম হয়; সাধারণত অনেকগুলো ছোট ছোট সমস্যা একসাথে মিলে বড় দুর্ঘটনা ঘটায় (যেমন Therac-25 case-এ দেখব)।
3. ভালো **software development procedure** মেনে চললে **errors** কমানো যায় — যেমন proper testing, code review, documentation।
4. **Code** ছোট রাখলে **confidence** বাড়ে — বড় কোডবেস বোঝা কঠিন, তাই ছোট, modular কোড লিখলে ভুল কম হয় এবং verify করাও সহজ হয়।

## What can go wrong?

- Roles
  - Computer user
    - Should understand limitations of computers
    - Should understand need for proper training and responsible use
  - Computer professional (buyer, developer, system manager...)
    - Needs to understand the sources and consequences of failures
  - Educated member of society
    - Understanding computer risks helps in discussing and deciding public policy.

### ২.২ তিনটা ভূমিকা (Roles) — কে কেন এই বিষয়ে সচেতন থাকা উচিত

সংজ্ঞা অনুযায়ী তিনটা role:

1. **Computer User** (ব্যবহারকারী):
    - কম্পিউটারের **limitations** বুঝতে হবে — কম্পিউটার সবসময় সঠিক ফলাফল দেয় না।
    - Proper training এবং responsible use-এর প্রয়োজনীয়তা বুঝতে হবে।
  2. **Computer Professional (buyer, developer, system manager):**
    - Failure-এর **sources** এবং **consequences** বুঝতে হবে — কোথা থেকে সমস্যা আসতে পারে এবং সেটার ফলাফল কী হতে পারে।
  3. **Educated member of society** (শিক্ষিত নাগরিক):
    - Computer risk বুঝলে **public policy** নিয়ে আলোচনা ও সিদ্ধান্ত নিতে সাহায্য হয় — যেমন AI regulation, data privacy law ইত্যাদি নিয়ে informed decision নেওয়া যায়।
-

# What can go wrong?

- Categories of computer errors and failures
  - Problems for individuals
    - affects one or a few people
  - System Failures
    - affects large numbers of people or costs large amounts of money or both
    - classic example: telecommunications network
  - Safety-Critical Applications
    - where property may be damaged or destroyed
    - where people may be injured or killed

## ৩. Categories of Computer Errors and Failures (৩টা ক্যাটাগরি — খুবই গুরুত্বপূর্ণ)

### ৩.১ Problems for Individuals (ব্যক্তিগত সমস্যা)

- সংজ্ঞা: এক বা কয়েকজন মানুষকে প্রভাবিত করে।
- উদাহরণ: billing error, database-এ ভুল তথ্য।

### ৩.২ System Failures (সিস্টেম ব্যর্থতা)

- সংজ্ঞা: অনেক মানুষকে প্রভাবিত করে অথবা অনেক টাকার ক্ষতি করে (বা দুটোই)।
- Classic example: **telecommunications network** ব্যর্থ হলে হাজার হাজার মানুষ প্রভাবিত হয়।

### ৩.৩ Safety-Critical Applications (নিরাপত্তা-সংকটাপন্ন প্রয়োগ)

- সংজ্ঞা: যেখানে সম্পত্তির ক্ষতি হতে পারে অথবা মানুষ আহত/নিহত হতে পারে।
- এটাই সবচেয়ে বিপজ্জনক ক্যাটাগরি — এখানেই Therac-25 এবং Patriot Missile-এর মতো কেস আসে।

মনে রাখার সহজ **Trick**: Severity বাড়ার ক্রম → Individual Problem (কম গুরুতর) → System Failure (মাঝারি) → Safety-Critical (সবচেয়ে গুরুতর, জীবন-মৃত্যুর প্রশ্ন)।

---

# What can go wrong?

- Problems for Individuals
  - Billing errors
    - Lack of tests for inconsistencies and inappropriate amounts
    - Perhaps caused by data entry errors due to poor HCI
  - Database accuracy problems
    - Incorrect information resulting in wrongful treatment or acts
  - Causes
    - large, diverse population
    - human common-sense not part of automated processing
    - overconfidence in accuracy of data from a computer
    - information not updated or corrected
    - lack of accountability for errors
- Q: How accurate should computer systems be?

## 8. Problems for Individuals — বিস্তারিত

ধরন:

1. **Billing errors** — ভুল বিল আসা। কারণ: inconsistency check না থাকা, অথবা poor HCI (Human-Computer Interaction)-এর কারণে ভুল data entry।
2. **Database accuracy problems** — ডাটাবেজে ভুল তথ্য থাকার কারণে ভুল treatment বা ভুল সিদ্ধান্ত নেওয়া হয়। (উদাহরণ: credit report-এ ভুল তথ্যের কারণে loan না পাওয়া)

### Causes (কারণসমূহ):

1. **Large, diverse population** — এত বিশাল ও বৈচিত্র্যময় জনগোষ্ঠীর ডাটা ম্যানেজ করা কঠিন, edge case মিস হয়ে যায়।
2. **Human common-sense automated processing**-এ নেই — কম্পিউটার যান্ত্রিকভাবে ডাটা প্রসেস করে, মানুষের মতো "এটা তো ভুল মনে হচ্ছে" — এই বোধ থাকে না।
3. **Computer** থেকে পাওয়া **data**-র উপর অতিরিক্ত আস্থা/বিশ্বাস (**overconfidence**) — মানুষ ধরে নেয় কম্পিউটার সবসময় ঠিক বলে।
4. তথ্য আপডেট/সংশোধন না করা — পুরনো ভুল তথ্য বছরের পর বছর থেকে যায়।
5. **Errors**-এর জন্য **accountability**-র অভাব — কেউ দায়িত্ব নেয় না ভুল সংশোধনের জন্য।

গুরুত্বপূর্ণ প্রশ্ন (স্লাইডে ছিল): "How accurate should computer systems be?" — এটা একটা open-ended discussion প্রশ্ন, উত্তর নির্ভর করে application-এর উপর (medical system-এ ১০০% accuracy দরকার, কিন্তু recommendation system-এ সেটা দরকার নেই)।

# What can go wrong?

- **System failures**
  - Communications
    - Telephone, online, and broadcast services
    - Cellular, paging, long-distance network
    - Broadband outages
  - Business
    - Inventory and management software
  - Financial
    - Stock exchange, brokerages, banks, etc.
  - Transportation
    - Reservations, ticketing, and baggage handling
  - Causes
    - Insufficient testing and debugging time
    - Significant changes in specifications (during and after project start)
    - Overconfidence in system
    - Mismanagement of the project

## ৫. System Failures — বিস্তারিত

কোন কোন **sector-এ System Failure** হয়:

1. **Communications** — টেলিফোন, অনলাইন, broadcast service, cellular, paging, long-distance network, broadband outage।
2. **Business** — inventory এবং management software ব্যর্থ হলে ব্যবসার কার্যক্রম বন্ধ হয়ে যায়।
3. **Financial** — stock exchange, brokerage, bank সিস্টেম fail করলে বিশাল আর্থিক ক্ষতি হয়।
4. **Transportation** — reservation, ticketing, baggage handling সিস্টেম fail করলে হাজার হাজার যাত্রী আটকে যায়।

**Causes (কারণ):**

1. **Insufficient testing and debugging time** — সময়ের চাপে পর্যাপ্ত টেস্ট না করেই সিস্টেম লঞ্চ করা।
  2. **Significant changes in specifications** — প্রজেক্ট চলাকালীন বা শুরুর পরে requirements বার বার বদলানো (scope creep)।
  3. **Overconfidence in system** — ডেভেলপার/ম্যানেজমেন্ট মনে করে সিস্টেম যথেষ্ট ভালো, তাই আরও টেস্টিং দরকার নেই।
  4. **Mismanagement of the project** — প্রজেক্ট ঠিকমতো পরিচালনা না হওয়া, deadline miss হওয়া, দুর্বল communication।
-

# What can go wrong?

- Safety-Critical Applications

- Uses

- Medicine, health-services
    - Power plants
    - Aircraft
    - Trains
    - Automated Factories
    - Military applications

- Causes of error:

- Overconfidence
    - Lack of override features
    - Insufficient testing
    - Poor HCI
    - Sheer complexity of system (both computer and environment)



University of Victoria

SENG 401: Social and Professional Issues

## ৬. Safety-Critical Applications — বিস্তারিত

কোথায় ব্যবহার হয়:

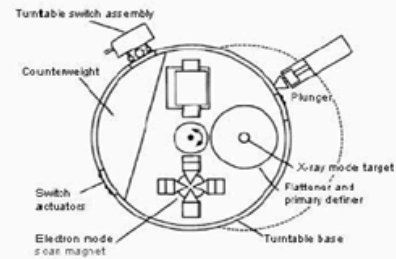
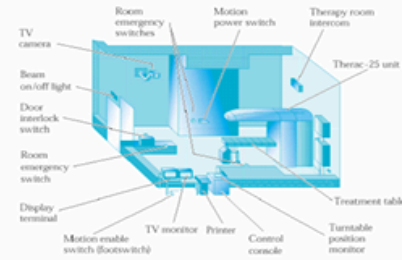
1. Medicine, health-services (যেমন রেডিওথেরাপি মেশিন)
2. Power plants (পারমাণবিক/বিদ্যুৎকেন্দ্র)
3. Aircraft (বিমান)
4. Trains
5. Automated Factories
6. Military applications (সামরিক ব্যবস্থা)

**Error-এর কারণসমূহ:**

1. **Overconfidence** — ডেভেলপার/অপারেটর মনে করে সিস্টেম fail করবে না।
  2. **Lack of override features** — মানুষ manually কন্ট্রোল নিয়ে সিস্টেম থামাতে না পারা।
  3. **Insufficient testing** — পর্যাপ্ত পরীক্ষা না করা, বিশেষ করে edge case-গুলো।
  4. **Poor HCI (Human-Computer Interaction)** — খারাপ ইন্টারফেস ডিজাইনের কারণে অপারেটর ভুল বোঝে বা ভুল command দেয়।
  5. **Sheer complexity of system** (কম্পিউটার এবং পরিবেশ দুটোই) — সিস্টেম এত জটিল হয়ে যায় যে কেউ পুরোপুরি বুঝতে পারে না।
-

# Case Study: Therac-25

- Produced by the Atomic Energy of Canada Limited (AECL)
- Introduced in 1983 as a successor model to the Therac-6 and Therac-20
- This new model provided full computer control, which itself was intended to provide several benefits
  - Faster setup by operators, hence more treatments per day.
  - Software checks replaced earlier hardware interlocks



University of Victoria  
Department of Computer Science

SENG 401: Social and Professional Issues  
Trusting Computers: Slide 13

## ১. Radiotherapy (রেডিওথেরাপি)

রেডিওথেরাপি হলো ক্যান্সারের চিকিৎসার একটি অন্যতম পদ্ধতি। এতে উচ্চ মাত্রার বিকিরণ (Radiation/Ray) ব্যবহার করে মানবদেহের ক্যান্সার কোষগুলোকে ধ্বংস করা হয় বা তাদের বৃদ্ধি থামিয়ে দেওয়া হয়।

- কীভাবে কাজ করে: চিকিৎসায় ব্যবহৃত যন্ত্রগুলো সাধারণত দুটি মোডে কাজ করে—
  1. **Low-Energy Mode (Electron Beam):** স্বকের উপরিভাগের ক্যান্সারের জন্য কম শক্তির রশ্মি।
  2. **High-Energy Mode (X-ray Beam):** শরীরের ভেতরের ক্যান্সারের জন্য অত্যন্ত শক্তিশালী রশ্মি (যা সরাসরি দিলে শরীর পুড়ে যাবে, তাই এটি একটি ধাতব ফিল্টার বা Target-এর মাধ্যমে দিয়ে নিরাপদ মোডে আনা হয়)।
- ঝুঁকি: এতে ব্যবহৃত রশ্মি অত্যন্ত শক্তিশালী। মাত্রা একটু ভুল হলেই রোগীর সুস্থ টিস্যু পুড়ে যেতে পারে বা মারাত্মক অঙ্গহানি হতে পারে।

## ২. Therac-25 (থেরাক-২৫ দুর্ঘটনা)

**Therac-25** ছিল ১৯৮০-এর দশকে তৈরি একটি কম্পিউটার-নিয়ন্ত্রিত রেডিওথেরাপি মেশিন (যা ক্যান্সার রোগীদের চিকিৎসায় ব্যবহৃত হতো)। এটি ছিল ইতিহাসে সফটওয়্যার ত্রুটির কারণে ঘটা অন্যতম ভয়াবহ দুর্ঘটনা।

ঘটনাটি কী ঘটেছিল?



১৯৮৫ থেকে ১৯৮৭ সালের মধ্যে এই মেশিনের সফটওয়্যার বাগের (Race Condition) কারণে অন্তত ৬ জন রোগীকে প্রয়োজনের চেয়ে ১০০ গুণ বেশি মাত্রার রেডিয়েশন (বিকিরণ) দেওয়া হয়। এর ফলে ৫ জন রোগী যন্ত্রণাদায়কভাবে মারা যান এবং অন্যরা মারাত্মক বিকলাঙ্গতার শিকার হন।

কেন এই ব্যর্থতা ঘটেছিল?

- **Software Bug (Race Condition):** দ্রুত টাইপ করে কমান্ড পরিবর্তন করলে মেশিনটির সফটওয়্যার বিভ্রান্ত হয়ে যেত। ইউজার বা ডাক্তার দ্রুত "X-ray Mode" থেকে "Electron Mode"-এ সুইচ করলে মেকানিক্যাল ফিল্টারটি সঠিক জায়গায় পৌঁছানোর আগেই মেশিন লেজার ফায়ার করে ফেলত।
- **Hardware Safety** তুলে নেওয়া: Therac-25-এর আগের মডেলে (Therac-20) হার্ডওয়্যারভিত্তিক সেফটি লক বা সুইচ ছিল। কিন্তু Therac-25-এ থরচ কমাতে এবং সফটওয়্যারের ওপর অতিরিক্ত আশ্রয়বিশ্বাসের কারণে সেই ব্যাকআপ হার্ডওয়্যার লকগুলো সরিয়ে নেওয়া হয়েছিল।
- অ্যালার্মের অস্পষ্টতা: যখনই কোনো ত্রুটি হতো, স্ক্রিনে শুধু একটি সাধারণ এরর কোড (যেমন: **MALFUNCTION 54**) দেখাত, যা দেখে ডাক্তাররা বুঝতে পারতেন না যে রোগী মারাত্মক রেডিয়েশনের শিকার হচ্ছেন।

### ৩. Patriot Missile Failure (প্যাট্রিওট ক্ষেপণাস্রের ব্যর্থতা)

১৯৯১ সালের উপসাগরীয় যুদ্ধের (Gulf War) সময় সৌদি আরবের ধারানে আমেরিকান **Patriot Missile System** একটি ইরাকি 'স্কাড' (Scud) ক্ষেপণাস্রকে আকাশে ধ্বংস করতে ব্যর্থ হয়। প্যাট্রিওট মিসাইলটি ঠিক সময়ে ফায়ার হয়নি।

এর ফলে স্কাড মিসাইলটি আমেরিকান সেনাশিবিরে আঘাত হানে এবং ২৮ জন মার্কিন সেনা নিহত ও ১০০ জনেরও বেশি আহত হন।

কেন এই ব্যর্থতা ঘটেছিল?

- **Floating-Point Rounding Error** (গাণিতিক ত্রুটি): মিসাইল সিস্টেমটি ভেতরে সময় হিসাব করার জন্য একটি ইন্টারনাল ক্লক (Internal Clock) ব্যবহার করত। ১ সেকেন্ডের দশ ভাগের এক ভাগকে (\$0.1\$ second) বাইনারিতে রূপান্তর করার সময় পয়েন্টের পরের সংখ্যার অতি ক্ষুদ্র রাউন্ডিং এরর (Rounding Error) তৈরি হতো।
- অতিরিক্ত সময় ধরে চালানো: সিস্টেমটি তৈরি করা হয়েছিল কয়েক ঘণ্টা চালু রেখে ব্যবহারের জন্য। কিন্তু ঘটনার দিন সিস্টেমটি একটানা ১০০ ঘণ্টা চালু ছিল।
- ক্ষুদ্র ত্রুটির বিশাল রূপ: প্রতি সেকেন্ডের সামান্য গাণিতিক ভুল ১০০ ঘণ্টা পর জমে জমে প্রায় ০.৩৪ সেকেন্ডের টাইমিং পার্থক্যে দাঁড়ায়।
- প্রচণ্ড গতিতে ধেয়ে আসা স্কাড মিসাইলের জন্য ০.৩৪ সেকেন্ড অনেক বড় সময়! এই সামান্য সময়ের ভুলেই প্যাট্রিওট মিসাইলটি স্কাড মিসাইলের সঠিক অবস্থান নির্ণয় করতে পারেনি এবং সেটিকে আকাশে মিস করে (এটি আকাশে স্কাডের অবস্থান থেকে প্রায় ৬০০ মিটার দূরে গিয়ে খুঁজেছিল)।

### সংক্ষিপ্ত সারসংক্ষেপ

| বিষয় | মূল ক্ষেত্র | মূল কারণ/সমস্যা | ফলাফল |
|-------|-------------|-----------------|-------|
|       |             |                 |       |

|                        |                                    |                                                         |                                                       |
|------------------------|------------------------------------|---------------------------------------------------------|-------------------------------------------------------|
| <b>Radiotherapy</b>    | চিকিৎসা প্রযুক্তি                  | ক্যান্সার কোষ ধ্বংসে ব্যবহৃত শক্তিশালী বিকিরণ।          | সঠিকভাবে ব্যবহার না হলে মারাত্মক ঝুঁকিপূর্ণ।          |
| <b>Therac-25</b>       | সফটওয়্যার ইঞ্জিনিয়ারিং ও চিকিৎসা | <b>Race Condition</b> এবং হার্ডওয়্যার সেফটি লকের অভাব। | রোগীদের ১০০ গুণ বেশি রেডিয়েশন প্রদান, ৫ জনের মৃত্যু। |
| <b>Patriot Missile</b> | সামরিক সফটওয়্যার                  | <b>Rounding Error</b> এবং দীর্ঘ সময় সিস্টেম চালু রাখা। | মিসাইল ইন্টারসেপ্ট করতে ব্যর্থতা, ২৮ জন সেনা নিহত।    |

## ৭. Case Study 1: Therac-25

### Background:

- **Atomic Energy of Canada Limited (AECL)** কোম্পানি তৈরি করেছিল।
- ১৯৮৩ সালে introduce হয়, আগের মডেল Therac-6 ও Therac-20-এর successor হিসেবে।
- এটা একটা **radiotherapy machine** — ক্যান্সার রোগীদের রেডিয়েশন থেরাপি দেওয়ার জন্য ব্যবহার হতো।

### নতুন মডেলের বিশেষত্ব:

- **Full computer control** — আগের হার্ডওয়্যার-ভিত্তিক সিস্টেমের বদলে পুরোপুরি সফটওয়্যার-নিয়ন্ত্রিত করা হয়।
- এর সুবিধা ছিল বলে ভাবা হয়েছিল:
  1. অপারেটরদের জন্য দ্রুত **setup** — তাই দিনে বেশি রোগী চিকিৎসা করা যায়।
  2. আগের হার্ডওয়্যার ইন্টারলক-এর (hardware interlock) জায়গায় **software check** বসানো হয়।

**"Hardware interlock"** মানে কী (সহজ **definition**): এটা একটা physical safety mechanism, যেমন একটা physical switch/lock, যা নিশ্চিত করে যে মেশিনের দুইটা অবস্থা একসাথে ঘটতে না পারে (যেমন high-power beam এবং no-target একসাথে চালু না হওয়া)। Therac-25-এ এই physical protection সরিয়ে শুধু software-এর উপর নির্ভর করা হয়েছিল — এটাই মূল সমস্যার একটা বড় কারণ।

### Radiotherapy সম্পর্কে সংক্ষিপ্ত ধারণা (**Detour slide** থেকে):

- **External Beam Therapy**-এর মূল আইডিয়া: টিউমার ধ্বংস করার জন্য একটা নির্দিষ্ট জায়গায় energy ফোকাস করা।

- আশেপাশের সুস্থ টিস্যুর ক্ষতি কমাতে beam রোগীর চারপাশে "rotate" করে।
- প্রক্রিয়া: **Simulation** → **Treatment planning** → **Treatment delivery**
- জড়িত থাকে: radiation oncologist (ডাক্তার), radiation physicist, dosimetrist, radiation therapist।
- **Radiation therapist** মেশিন চালান, কিন্তু বিম/ডোজ আগে থেকেই পরিকল্পনা করা থাকে।
- **Linear accelerator** বা **cobalt machine** দিয়ে রেডিয়েশন তৈরি হয়।

## দুর্ঘটনার বিবরণ:

1. দুই বছরের মধ্যে ৬ জন রোগী overdose-এর শিকার হন।
2. প্রথম ঘটনা ঘটে **Hamilton, Ontario**-তে।
3. স্বাভাবিক রেডিয়েশন ডোজ: ১০০-২০০ **rads**।
4. তুলনা করার জন্য (কতটা কম ডোজ দৈনন্দিন medical imaging-এ ব্যবহার হয়):
  - Dental x-ray: 0.02 rads
  - Chest x-ray (দুই view): 0.02-0.07 rads
  - CT scan (মাথা/বুক): 1 rad
5. Therac-25-এর overdose ছিল ১৩,০০০ থেকে ২৫,০০০ **rads** — মানে স্বাভাবিক ডোজের প্রায় ১০০-২০০ গুণ বেশি!
6. ৬ জনের মধ্যে ৩ জন মারা যান।

## ব্যর্থতার একাধিক কারণ (Multiple, interrelated causes) — এটা মুখস্থ রাখা জরুরি:

1. **Poor safety design** — নিরাপত্তার দিক থেকে দুর্বল ডিজাইন থেকেই সমস্যার শুরু।
2. **Insufficient testing and debugging** — পর্যাপ্ত পরীক্ষা করা হয়নি।
3. **Software errors** — কোডে race condition-এর মতো bug ছিল (অপারেটর দ্রুত data এডিট করলে সিস্টেম ভুল mode-এ চলে যেত)।
4. **Lack of safety interlocks (hardware)** — আগের মডেলে থাকা physical safety mechanism বাদ দেওয়া হয়েছিল।
5. **Overconfidence** — নির্মাতারা মনে করেছিল সফটওয়্যার নিজেই যথেষ্ট নিরাপদ, তাই hardware backup দরকার নেই।
6. **Inadequate reporting/investigation of accidents** — প্রথম দুর্ঘটনার পরেও সঠিকভাবে তদন্ত করা হয়নি, তাই একই সমস্যা বার বার ঘটেছিল অন্য জায়গায়।

## পরিণতি:

- Therac-25 মেশিন সম্পূর্ণ বন্ধ করা হয়নি, বরং **hardware interlock** আবার বসিয়ে (**retrofit** করে) ব্যবহার চালু রাখা হয়েছিল — এটা প্রমাণ করে যে hardware safety backup কতটা গুরুত্বপূর্ণ, শুধু software-এর উপর নির্ভর করা যায় না।

**Therac-25-এর মূল শিক্ষা (Key Takeaway):** শুধু software-নির্ভর সিস্টেমে physical/hardware backup safety mechanism অবশ্যই থাকা উচিত — কারণ software-এ bug থাকতেই পারে, কিন্তু hardware interlock সেই bug-কে বাস্তব জগতে ক্ষতি করা থেকে আটকাতে পারে।

---

## Case Study: Patriot Missile Failure

- February 1991, During first Gulf War
  - Patriot Missile battery in Dharran, Saudi Arabia, failed to intercept incoming Iraqi Scud missile
  - Scud struck an American Army barracks
  - 28 soldiers killed
- Reported cause of failure
  - Inaccurate calculation of “time since boot”
  - Due to computer-arithmetic errors

## ৮. Case Study 2: Patriot Missile Failure (গুরুত্বপূর্ণ, technical detail সহ)

### ঘটনার বিবরণ:

1. ফেব্রুয়ারি ১৯৯১, First Gulf War চলাকালীন।
2. Saudi Arabia-এর **Dharran**-এ থাকা Patriot Missile battery একটা আসা Iraqi **Scud missile** ঠেঁকাতে ব্যর্থ হয়।
3. Scud missile একটা **American Army barracks**-এ আঘাত করে।
4. ২৮ জন সৈনিক নিহত হন।

### রিপোর্ট করা ব্যর্থতার কারণ:

- “Time since boot”-এর ভুল হিসাব — সিস্টেম কতক্ষণ ধরে চালু আছে তার হিসাবে ভুল হয়েছিল।
- এটা হয়েছিল **computer arithmetic error**-এর কারণে।

### Technical Details (এটা ডিটেইলে বোঝা দরকার, কারণ এটা একটা classic উদাহরণ কীভাবে ছোট numeric error বড় বিপর্যয় ঘটাতে পারে):

1. সিস্টেমের internal clock সময় মাপত ১/১০ সেকেন্ড এককে — যেমন, ১০০০ ইউনিট মানে ১০০ সেকেন্ড।
2. সেকেন্ডে সময় বের করার জন্য এই ইউনিটগুলোকে ১/১০ দিয়ে গুণ করা হতো।
3. এই গুণ করা হতো একটা **24-bit fixed-point register**-এ।

4. সমস্যা হলো, **1/10**-এর বাইনারি সম্প্রসারণ (**binary expansion**) হলো: 0.00011001100110011...  
— এটা একটা অসীম পুনরাবৃত্তিমূলক (**repeating**) বাইনারি সংখ্যা (ঠিক যেমন দশমিকে  $1/3 = 0.333...$  অসীম চলতে থাকে)।
5. যেহেতু 24-bit register-এ সসীম জায়গা আছে, তাই এই সংখ্যাটা **truncate** (কেটে ফেলা) করতে হয় — এটাই মূল ত্রুটির উৎস।
6. সিস্টেম যখন ১০০ ঘন্টা ধরে চালু ছিল, তখন এই ছোট truncation error জমতে জমতে বড় হয়ে যায়।
7. মোট error দাঁড়ায় **0.34** সেকেন্ড।
8. Scud missile-এর গতি: প্রতি সেকেন্ডে ১,৬৭৬ মিটার।
9. তাই 0.34 সেকেন্ডের ভুল মানে দাঁড়ায় প্রায় আধা কিলোমিটার (**0.5 km**) দূরত্বের ভুল হিসাব — এই কারণে Patriot missile-এর tracking system Scud missile-টাকে সঠিক জায়গায় খুঁজে পায়নি এবং intercept করতে ব্যর্থ হয়।

মূল শিক্ষা (**Key Takeaway**): এত ছোট একটা numeric/rounding error (মাত্র 0.34 সেকেন্ড!) যদি সিস্টেম দীর্ঘক্ষণ চালু থাকে (accumulate হতে থাকে), তাহলে সেটা বাস্তব জীবনে ভয়াবহ পরিণতি ডেকে আনতে পারে। এটা প্রমাণ করে যে **numeric precision** এবং **floating-point/fixed-point arithmetic**-এর সীমাবদ্ধতা সফটওয়্যার ইঞ্জিনিয়ারিং-এ কতটা গুরুত্বপূর্ণ বিষয়।

## Interface Design Perspective

- To the user “the interface is the system”
  - Understanding of the system operation is based on its interface
  - If this conceptual model is wrong, errors result
- Human attention is incomplete – will not notice every detail, every notification
- Good design will help prevent human error and help users recover from errors

## ৯. Interface Design Perspective (ইন্টারফেস ডিজাইনের দৃষ্টিকোণ)

মূল ধারণা:

1. **"To the user, the interface is the system"** — ব্যবহারকারীর কাছে ইন্টারফেসটাই পুরো সিস্টেম, কারণ সে সিস্টেম কীভাবে কাজ করে সেটা শুধু ইন্টারফেস দেখেই বোঝে (ভেতরের কোড দেখে না)।
2. ব্যবহারকারীর সিস্টেম সম্পর্কে একটা **conceptual model** (মানসিক ধারণা) তৈরি হয় ইন্টারফেস দেখে। যদি এই ধারণাটা ভুল হয়, তাহলে **errors** ঘটে।
3. মানুষের মনোযোগ অসম্পূর্ণ (**incomplete**) — কোনো মানুষ প্রতিটা detail বা প্রতিটা notification খেয়াল করবে না। এটা মানুষের স্বাভাবিক সীমাবদ্ধতা।
4. ভালো ডিজাইন **human error** প্রতিরোধ করতে সাহায্য করে, এবং error হয়ে গেলে সেখান থেকে recover করতেও সাহায্য করে।

## "Response Chaining" — একটা গুরুত্বপূর্ণ concept

সংজ্ঞা: Response chaining মানে হলো, বার বার একই ধরনের action (যেমন "Enter" বাটন চাপা) করতে করে সেই action **automatic/mechanical** হয়ে যায় — মানুষ তখন আর প্রতিটা স্ক্রিনের কনটেন্ট মনোযোগ দিয়ে পড়ে না, শুধু যান্ত্রিকভাবে বাটন চাপতে থাকে।

উদাহরণ (স্লাইডে দেওয়া):

- একজন নার্স গুরুতর অসুস্থ রোগীর মনিটরিং সিস্টেমের alarm ভুলবশত বন্ধ করে দেন।
- স্বাভাবিক কাজ ছিল: alarm সেট করা, তারপর confirmation screen-এর একটা সিরিজের মধ্য দিয়ে "Enter" চেপে চেপে এগিয়ে যাওয়া।
- এই action এতটাই routine/automatic (tap tap tap) হয়ে গিয়েছিল যে, alarm বন্ধ হয়ে গেছে — এই warning-টা নার্স খেয়ালই করেননি।

এখান থেকে শিক্ষা: Repetitive confirmation screen ডিজাইন করার সময় সতর্ক থাকা উচিত, কারণ মানুষ বার বার একই কাজ করলে সেটা automatic হয়ে যায় এবং গুরুত্বপূর্ণ warning মিস হয়ে যেতে পারে।

---

## What goes wrong?

- Computer systems fail because:
  - The job they are doing is inherently difficult
  - The job is sometimes done poorly
- Compounding the reliability issue
  - Developers and users are overconfident in the system
  - Re-used system software may not work correctly in different environments
- Intellectual overload
  - Large bodies of code are more difficult to understand than smaller ones

## ১০. What Goes Wrong? — সামগ্রিক বিশ্লেষণ

কম্পিউটার সিস্টেম কেন ব্যর্থ হয়:

1. কাজটা স্বভাবতই কঠিন (**inherently difficult**) — কিছু সমস্যা এতটাই জটিল যে নিখুঁতভাবে সমাধান করা কঠিন।
2. কাজটা কখনো কখনো খারাপভাবে করা হয় — মানবিক ভুল, তাড়াহুড়া, অপরিপািত সম্পদ।

**Reliability** সমস্যা আরও বাড়িয়ে দেয়:

3. **Developer** এবং **user** দুজনেরই সিস্টেমের প্রতি অতিরিক্ত আত্মবিশ্বাস (**overconfidence**) থাকে।
4. পুনরায় ব্যবহৃত (**re-used**) সিস্টেম সফটওয়্যার ভিন্ন পরিবেশে (**environment**) সঠিকভাবে কাজ নাও করতে পারে — একটা কনটেক্সটে ভালো কাজ করা কোড অন্য কনটেক্সটে সমস্যা তৈরি করতে পারে।

**Intellectual Overload** (বৌদ্ধিক অতিরিক্ত চাপ):

5. বড় কোডবেস বোঝা ছোট কোডবেসের চেয়ে অনেক বেশি কঠিন — মানুষের মস্তিষ্কের একসাথে সীমিত পরিমাণ জটিলতা ধরে রাখার ক্ষমতা আছে, তাই বড় সিস্টেমে bug থাকার সম্ভাবনা বেশি এবং সেটা খুঁজে পাওয়াও কঠিন।
-

Q: How can we increase reliability & safety?

- As developers?
- As system managers?
- As users?

## প্রশ্ন: কীভাবে **Reliability & Safety** বাড়ানো যায়? (Developer, System Manager, User হিসেবে) — ১০-১২ পয়েন্ট

স্লাইডে এই প্রশ্নটা করা হয়েছে (৩টা দৃষ্টিকোণ থেকে চিন্তা করতে বলা হয়েছে), এবং এর উত্তর পরের স্লাইডগুলোতেই দেওয়া আছে। নিচে সেটা সম্পূর্ণ ডিটেইলে দিচ্ছি:

### A) Professional Responsibility (পেশাগত দায়িত্ব):

1. ভালো **software-engineering practice** অনুসরণ করা — যেমন version control, code review, documentation।
2. ভালো ডিজাইন করা **user interface** তৈরি করা এবং **human factors** বিবেচনায় রাখা (মানুষ কীভাবে ভুল করতে পারে সেটা মাথায় রেখে ডিজাইন করা)।
3. **Built-in redundancy** রাখা — একটা component fail করলে যেন backup system কাজ চালিয়ে যেতে পারে (যেমন Therac-25-এ hardware interlock থাকলে software bug হলেও সুরক্ষা থাকত)।
4. যথাযথ জায়গায় **self-checking mechanism** যুক্ত করা — সিস্টেম নিজে নিজে তার output validate করতে পারবে এমন ব্যবস্থা রাখা।
5. ভালো **testing principle** এবং **technique** অনুসরণ করা — unit testing, integration testing, edge case testing ইত্যাদি।

### B) Law and Regulation (আইন ও নিয়মকানুন) দিয়ে:

6. **Civil** এবং **criminal penalty** — ত্রুটিপূর্ণ বা অনিরাপদ সিস্টেমের কারণে ক্ষতি হলে সেটা পুষিয়ে নেওয়ার জন্য, এবং নিরাপদ সিস্টেম তৈরি করার জন্য প্রণোদনা (incentive) দেওয়ার জন্য।
7. **Warranty** — একটা নির্দিষ্ট মানের গ্যারান্টি দেওয়ার জন্য।
8. **Provincial/Federal regulation** — যেমন database accuracy regulation, যা প্রাইভেট কোম্পানি ও সরকারের রাখা ভুল তথ্য থেকে জনগণকে রক্ষা করে।
9. **Software developer**-দের জন্য বাধ্যতামূলক লাইসেন্সিং — যাতে proper training, competency, এবং continuing education নিশ্চিত করা যায় (ঠিক যেমন ডাক্তার/ইঞ্জিনিয়ারদের লাইসেন্স লাগে)।

### C) User এবং System Manager হিসেবে:

10. **Proper training** দেওয়া — ব্যবহারকারীদের সিস্টেমের সীমাবদ্ধতা এবং সঠিক ব্যবহার শেখানো।
11. নিয়মিত **system audit** এবং **monitoring** — সিস্টেম ঠিকমতো কাজ করছে কিনা তা নিয়মিত যাচাই করা।
12. **Override/manual control**-এর ব্যবস্থা রাখা — জরুরি পরিস্থিতিতে মানুষ যেন সিস্টেমকে override করতে পারে (automated system পুরোপুরি অন্ধভাবে বিশ্বাস না করে)।



গুরুত্বপূর্ণ **Discussion** প্রশ্ন (স্লাইডে ছিল): "Mandatory licensing of software developers — Do you think this is a good idea?" এটা একটা open debate:

- পক্ষে যুক্তি: এটা মান নিয়ন্ত্রণ নিশ্চিত করে, ঠিক যেমন ডাক্তার/সিভিল ইঞ্জিনিয়ারদের লাইসেন্স লাগে (কারণ তাদের ভুলে মানুষ মারা যেতে পারে)।
- বিপক্ষে যুক্তি: সফটওয়্যার ইন্ডাস্ট্রি অনেক দ্রুত পরিবর্তনশীল (fast-evolving), তাই কড়া লাইসেন্সিং innovation-কে বাধা দিতে পারে এবং entry barrier তৈরি করতে পারে।

- Mandatory licensing of software developers
  - to ensure proper training, competency, and continuing education
  - Q: Do you think this is a good idea?

## Mandatory Licensing of Software Developers — উত্তর

মূল উত্তর: Software developer-দের জন্য বাধ্যতামূলক (mandatory) লাইসেন্সিং একটা বিতর্কিত (debatable) প্রস্তাব — এর লক্ষ্য হলো proper training, competency, এবং continuing education নিশ্চিত করা (ঠিক যেমন ডাক্তার বা সিভিল ইঞ্জিনিয়ারদের ক্ষেত্রে হয়), কিন্তু এটা বাস্তবায়ন করা সহজ না, এবং এর পক্ষে-বিপক্ষে শক্তিশালী যুক্তি আছে। তাই এর "ভালো/খারাপ" নির্ধারণ পুরোপুরি প্রেক্ষাপট (context) ও প্রয়োগের ধরনের উপর নির্ভর করে।

নিচে বিস্তারিতভাবে পয়েন্ট আকারে বিশ্লেষণ করা হলো:

### পক্ষে যুক্তি (Arguments For Mandatory Licensing):

1. মান নিয়ন্ত্রণ (**Quality control**) নিশ্চিত হবে: ঠিক যেমন ডাক্তার বা সিভিল ইঞ্জিনিয়ারদের লাইসেন্স লাগে (কারণ তাদের ভুলে মানুষ মারা যেতে পারে), তেমনি safety-critical software (যেমন Therac-25-এর মতো medical device, aircraft control system) তৈরির ক্ষেত্রেও একই যুক্তি প্রযোজ্য — যেখানে software error সরাসরি মানুষের জীবন ঝুঁকিতে ফেলতে পারে।
2. ন্যূনতম দক্ষতার (**baseline competency**) নিশ্চয়তা দেয়: লাইসেন্সিং নিশ্চিত করে যে একজন developer অন্তত মৌলিক software engineering principle, testing methodology, এবং নিরাপত্তা মানদণ্ড (safety standard) জানে — যেটা Therac-25 এবং Patriot Missile-এর মতো দুর্ঘটনা প্রতিরোধে সাহায্য করতে পারত।
3. **Continuing education** বাধ্যতামূলক করবে: প্রযুক্তি দ্রুত পরিবর্তনশীল, তাই লাইসেন্সের সাথে যুক্ত থাকা renewal প্রক্রিয়া developer-দের নতুন প্রযুক্তি ও নিরাপত্তা পদ্ধতি সম্পর্কে আপডেট থাকতে বাধ্য করবে।
4. **Accountability** (জবাবদিহিতা) সহজ হবে: যদি লাইসেন্সপ্রাপ্ত কেউ গুরুতর ভুল করে, তাহলে তার লাইসেন্স বাতিল/স্বগিত করা যাবে — এটা একটা স্পষ্ট জবাবদিহিতার কাঠামো তৈরি করে, যা এখন সফটওয়্যার ইন্ডাস্ট্রিতে অনুপস্থিত (Week 9-এর "lack of accountability" সমস্যার সাথে সরাসরি সম্পর্কিত)।

## বিপক্ষে যুক্তি (Arguments Against Mandatory Licensing):

5. প্রযুক্তি অত্যন্ত দ্রুত পরিবর্তনশীল (**fast-evolving field**): সফটওয়্যার ইন্ডাস্ট্রি এত দ্রুত পরিবর্তিত হয় যে একটা কঠোর, প্রাতিষ্ঠানিক licensing system তৈরি করতে করতেই সেটা পুরনো হয়ে যেতে পারে — যেখানে সিভিল ইঞ্জিনিয়ারিং-এর মূলনীতি (physics, structural integrity) কয়েক দশক ধরে প্রায় অপরিবর্তিত থাকে।
6. **Innovation**-এ বাধা (**barrier to entry**) তৈরি হতে পারে: অনেক সফল self-taught developer এবং startup founder আছেন যাদের কোনো formal degree নেই — বাধ্যতামূলক লাইসেন্সিং এই ধরনের মানুষদের ইন্ডাস্ট্রিতে প্রবেশ করতে বাধা দিতে পারে, যা innovation-এর জন্য ক্ষতিকর।
7. সব সফটওয়্যার সমান ঝুঁকিপূর্ণ না — **"one-size-fits-all"** সমস্যা: একটা simple website বানানো আর একটা medical device-এর software বানানো সম্পূর্ণ ভিন্ন ঝুঁকির মাত্রা বহন করে — সব ধরনের সফটওয়্যার development-এই একই কঠোর লাইসেন্সিং প্রয়োগ করা অযৌক্তিক এবং অপ্রয়োজনীয়ভাবে বোঝা তৈরি করতে পারে।
8. বাস্তবায়নের ব্যবহারিক জটিলতা: কে নির্ধারণ করবে লাইসেন্সিং পরীক্ষার মানদণ্ড কী হবে? কোন প্রোগ্রামিং ভাষা/framework-এর জ্ঞান আবশ্যিক? এত বিস্তৃত ও দ্রুত পরিবর্তনশীল একটা ক্ষেত্রে একটা নির্দিষ্ট পরীক্ষা তৈরি করা বাস্তবে অত্যন্ত কঠিন।

---

## একটা ভারসাম্যপূর্ণ (Balanced) সমাধান — মূল পয়েন্ট:

9. **Risk-based approach** (ঝুঁকিভিত্তিক পদ্ধতি) হতে পারে সমাধান: সব সফটওয়্যার developer-এর জন্য বাধ্যতামূলক লাইসেন্সিং না চাপিয়ে, শুধুমাত্র **safety-critical application** (medical device, aviation, nuclear power plant, autonomous vehicle software)-এর ক্ষেত্রে বাধ্যতামূলক লাইসেন্সিং প্রয়োগ করা যেতে পারে — যেখানে ভুলের পরিণতি জীবন-মৃত্যুর প্রশ্ন তৈরি করে।
10. সিদ্ধান্ত: সম্পূর্ণ ইন্ডাস্ট্রি-ব্যাপী বাধ্যতামূলক লাইসেন্সিং (সব ধরনের সফটওয়্যার development-এর জন্য) বাস্তবসম্মত বা কার্যকর নাও হতে পারে, কারণ এটা innovation-কে বাধাগ্রস্ত করতে পারে এবং প্রযুক্তির দ্রুত পরিবর্তনের সাথে তাল মিলিয়ে চলা কঠিন। কিন্তু **safety-critical software**-এর জন্য নির্দিষ্ট, সীমিত পরিসরের **licensing/certification** ব্যবস্থা — Week 9-এর "Increasing Reliability and Safety" আলোচনার সাথে সামঞ্জস্যপূর্ণ একটা যুক্তিসঙ্গত পদক্ষেপ হতে পারে, যা মান নিয়ন্ত্রণ এবং innovation-এর স্বাধীনতা — এই দুইয়ের মধ্যে একটা ভারসাম্য বজায় রাখে।

---

## ১১. Computer Models (কম্পিউটার মডেল)

### মূল বিষয় (Points to consider):

1. **Model** মানে হলো **simplification** — physical অথবা intangible সিস্টেমের একটা সরলীকৃত রূপ। এটা কখনোই বাস্তবতার হুবহু প্রতিক্রিয়া (exact replica) না।

2. যারা **model** তৈরি করে (**developer**) তাদের ফলাফল নিয়ে সৎ (honest) ও নির্ভুল (accurate) থাকতে হবে।
3. **Computer professional** এবং সাধারণ জনগণকে **developer**-দের দাবি (**claims**) মূল্যায়ন করার সক্ষমতা থাকতে হবে — মানে model-এর ফলাফলকে অন্ধভাবে বিশ্বাস না করে যাচাই করার ক্ষমতা থাকা উচিত।

## Model কেন সঠিক নাও হতে পারে (কারণসমূহ):

1. **Developer**-দের সিস্টেম সম্পর্কে অসম্পূর্ণ জ্ঞান (**incomplete knowledge**) — যে সিস্টেম model করা হচ্ছে সেটা সম্পর্কে developer-এর পুরোপুরি জ্ঞান না থাকা।
2. **Data** অসম্পূর্ণ বা ভুল হতে পারে — যদি input data-ই ভুল হয়, model-এর output-ও ভুল হবে (Garbage In, Garbage Out — GIGO principle)।
3. কম্পিউটারের ক্ষমতা অপরিাপ্ত হতে পারে — জটিল সিস্টেম model করার জন্য যথেষ্ট computational power না থাকা।
4. কিছু **variable** সংখ্যায় (**numerically**) প্রকাশ করা কঠিন — যেমন মানুষের আচরণ, সামাজিক প্রভাব ইত্যাদি সংখ্যায় রূপান্তর করা কঠিন।
5. রাজনৈতিক ও অর্থনৈতিক প্রেরণা ফলাফল বিকৃত করতে পারে (**political and economic motivation to distort results**) — যাদের স্বার্থ জড়িত, তারা model-এর ফলাফল নিজেদের পক্ষে দেখানোর চেষ্টা করতে পারে।

## দুইটা বিপরীতমুখী উদাহরণ (Two contrasting examples):

### Car-crash Models (গাড়ি দুর্ঘটনার মডেল):

মূল্যায়নের জন্য জিজ্ঞাসা করা উচিত:

1. Modeler-রা যে সিস্টেম বা material নিয়ে কাজ করছে সেটা কতটা ভালোভাবে বোঝে?
2. Data কতটা নির্ভুল ও সম্পূর্ণ?
3. Model-এর assumption এবং simplification কী কী?
4. Model-এর ফলাফল/prediction কি বাস্তব জগতের ফলাফলের সাথে মিলে যায়?

### Climate Models (জলবায়ু মডেল):

- একই ৪টা প্রশ্নই প্রযোজ্য।
  - তুলনামূলক পার্থক্য: Car-crash model তুলনামূলক ছোট, নিয়ন্ত্রিত (controlled) পরিবেশে যাচাই করা সহজ (physical crash test দিয়ে verify করা যায়)। কিন্তু Climate model বিশাল, জটিল, দীর্ঘমেয়াদী সিস্টেম নিয়ে কাজ করে, যেখানে হাজার হাজার variable interact করে এবং verify করা অনেক কঠিন (কারণ আমরা "বাস্তব পৃথিবী"-কে বার বার test করতে পারি না)।
-

# Perspectives

- Failures
  - What are “acceptable rates” of failures?
  - How accurate should software be?
- Dependence
  - How dependent on computer systems are our ordinary activities?
  - How useful are computer systems to our ordinary activities?
- Risks and progress
  - How do new technologies become safer?
  - Can progress in software safety keep up with the pace of change in computer technology?

## ১২. Perspectives on Failures, Dependence, Risk and Progress

### ৩টা দৃষ্টিকোণ (Perspective):

#### A) Failures (ব্যর্থতা নিয়ে প্রশ্ন):

1. "Acceptable rate of failure" কী হওয়া উচিত? — কোনো সিস্টেমই ১০০% নির্ভুল না, তাহলে কতটুকু ভুল acceptable?
2. Software কতটা নির্ভুল হওয়া উচিত? — এটা application-ভেদে পরিবর্তিত হয় (medical device vs. entertainment app)।

#### B) Dependence (নির্ভরতা নিয়ে প্রশ্ন):

3. আমাদের সাধারণ কার্যক্রম কম্পিউটার সিস্টেমের উপর কতটা নির্ভরশীল?
4. কম্পিউটার সিস্টেম আমাদের সাধারণ কার্যক্রমে কতটা উপযোগী?

#### C) Risks and Progress (ঝুঁকি এবং অগ্রগতি):

5. নতুন প্রযুক্তি কীভাবে ক্রমান্বয়ে নিরাপদ হয়ে ওঠে?
  6. Software safety-তে অগ্রগতি কি কম্পিউটার প্রযুক্তির পরিবর্তনের গতির সাথে তাল মিলিয়ে চলতে পারে?
-

## ১৩. Risk Is Not New (ঝুঁকি নতুন কিছু না)

### মূল ধারণা:

1. পুরনো প্রযুক্তিতেও ঝুঁকি ছিল — গাড়ি, বিমান, ট্রেন, বাষ্প ইঞ্জিন (steam engine), এলিভেটর — এসব প্রযুক্তি প্রথম দিকে অনেক দুর্ঘটনা ঘটিয়েছিল, কিন্তু সময়ের সাথে সাথে এগুলো নিরাপদ হয়ে উঠেছে।
2. কম্পিউটার ঝুঁকির ধরন এবং পরিধি বদলে দিয়েছে (**changes the types and scope of risk**), এবং দায় (**blame**) বন্টনের ধরনও বদলে দিয়েছে:
  - সিস্টেম আরও বড়, আরও জটিল হয়ে গেছে — একটা ছোট bug এখন বিশ্বব্যাপী প্রভাব ফেলতে পারে (interconnected systems)।
  - কম্পিউটার নিজেই সিদ্ধান্ত নেয় (**computers make decisions**) — আগে মানুষ সরাসরি সিদ্ধান্ত নিত, এখন automated/AI system সিদ্ধান্ত নেয়, তাই ভুল হলে দায় কার (মানুষের নাকি সিস্টেমের) সেটা নির্ধারণ করা কঠিন হয়ে যায়।

গুরুত্বপূর্ণ তুলনা: পুরনো প্রযুক্তির ঝুঁকি এবং কম্পিউটার ঝুঁকির মধ্যে মূল পার্থক্য হলো **scale** (পরিধি) এবং **decision-making autonomy** (স্বাধীন সিদ্ধান্ত গ্রহণ ক্ষমতা)। একটা গাড়ির ত্রুটি একজনকে প্রভাবিত করে, কিন্তু একটা ভুল software update হাজার হাজার/লক্ষ লক্ষ মানুষকে একসাথে প্রভাবিত করতে পারে (যেমন airline booking system crash)।

## ১৪. Summary / Revision Part (সব কিছু এক নজরে)

- ৩টা **Category of Error/Failure**: Individual Problem (কম গুরুতর) → System Failure (মারাত্মক) → Safety-Critical Application (সবচেয়ে গুরুতর)।
- ৩টা **Role**: User (limitation বুঝতে হবে), Professional (source/consequence বুঝতে হবে), Society member (policy-তে অংশগ্রহণের জন্য বুঝতে হবে)।
- **Therac-25 Case**: পুরোপুরি software-নির্ভর radiotherapy machine, hardware interlock বাদ দেওয়া হয়েছিল — ফলে ৬ জন overdose, ৩ জন মৃত্যু। মূল শিক্ষা: **software-only safety** যথেষ্ট না, **hardware backup** দরকার।
- **Patriot Missile Case**: 1/10-এর binary representation truncation error → ১০০ ঘন্টায় জমে ০.৩৪ সেকেন্ড ভুল → প্রায় আধা কিলোমিটার মিস → ২৮ জন নিহত। মূল শিক্ষা: ছোট **numeric/rounding error** দীর্ঘসময় ধরে জমতে জমতে বিশাল বিপর্যয় ঘটাতে পারে।
- **Interface Design**: "Interface is the system" (ব্যবহারকারীর কাছে)। **Response chaining** সমস্যা — বার বার একই action করলে সেটা automatic হয়ে যায় এবং গুরুত্বপূর্ণ warning miss হয়ে যায়।
- **Reliability** বাড়ানোর উপায়: ভালো software engineering practice, redundancy, self-checking, testing, আইন/regulation, licensing, training।
- **Computer Models**: সবসময় simplification, কখনোই নিখুঁত না — কারণ: অসম্পূর্ণ জ্ঞান, ভুল data, অপর্যাপ্ত computing power, quantify করা কঠিন variable, রাজনৈতিক/অর্থনৈতিক bias। Car-crash model বনাম Climate model — উভয়ই একই ৪টা মূল্যায়ন প্রশ্নের মুখোমুখি হয়।

- **Risk** পূরনো কিছু না, কিন্তু কম্পিউটার এর **scale** ও **decision-making autonomy** বদলে দিয়েছে — এখন সিস্টেম নিজেই সিদ্ধান্ত নেয়, তাই **accountability** (দায়বদ্ধতা) নির্ধারণ করা আরও জটিল।

---

## Can progress in software safety keep up with the pace of change in computer technology? — উত্তর

মূল উত্তর: না, বেশিরভাগ ক্ষেত্রেই software safety-তে অগ্রগতি computer technology-র পরিবর্তনের গতির সাথে সম্পূর্ণভাবে তাল মিলিয়ে চলতে পারে না — কারণ প্রযুক্তি সূচকীয় (exponential) হারে দ্রুত পরিবর্তিত হয়, কিন্তু safety practice, testing methodology, এবং regulation তৈরি হতে তুলনামূলক অনেক বেশি সময় লাগে। ফলে সবসময় একটা **"safety gap"** (নিরাপত্তা ফাঁক) থেকে যায়, যদিও সময়ের সাথে সাথে এই ফাঁক কমানোর চেষ্টা চলছে।

নিচে বিস্তারিতভাবে পয়েন্ট আকারে বিশ্লেষণ করা হলো:

---

কেন **Safety** প্রযুক্তির গতির সাথে তাল মেলাতে পারে না (পক্ষে যুক্তি):

1. প্রযুক্তির পরিবর্তনের গতি সূচকীয় (**exponential**): কম্পিউটার প্রযুক্তি (processing power, AI capability, connectivity) প্রতি কয়েক বছরে দ্বিগুণ বা তার বেশি হারে উন্নত হচ্ছে, কিন্তু safety testing ও verification প্রক্রিয়া তুলনামূলকভাবে ধীর ও শ্রমসাধ্য (labor-intensive) থেকে যায়।
2. নতুন প্রযুক্তি নতুন, অজানা ঝুঁকি নিয়ে আসে: যখন কোনো সম্পূর্ণ নতুন প্রযুক্তি (যেমন AI/machine learning, autonomous vehicle, IoT device) বাজারে আসে, তখন তার সম্ভাব্য ব্যর্থতার ধরন (failure mode) সম্পর্কে পর্যাপ্ত পূর্ব-অভিজ্ঞতা (historical data) থাকে না — safety practitioner-দের এই নতুন ঝুঁকি বুঝতেই সময় লাগে।
3. **Complexity** ক্রমাগত বাড়ছে (**Week 9-এর "Intellectual Overload" concept-এর সাথে** সরাসরি সম্পর্কিত): আধুনিক সফটওয়্যার সিস্টেম (যেমন self-driving car, বড় distributed system) এতটাই জটিল যে কোনো একজন মানুষ বা এমনকি একটা দল পুরো সিস্টেমটা সম্পূর্ণভাবে বুঝতে পারে না — যত জটিলতা বাড়ে, ততই ভুল/bug লুকিয়ে থাকার সম্ভাবনা বাড়ে।
4. আইন ও **regulation** তৈরি হতে অনেক সময় লাগে: সরকারি নীতি ও আইন প্রণয়ন প্রক্রিয়া স্বভাবতই ধীর (বিতর্ক, পরামর্শ, পাস করা) — যেমন Bill C-32-এর মতো copyright আইন পাস হতে বছরের পর বছর লেগেছিল, একইভাবে নতুন প্রযুক্তির জন্য safety regulation তৈরি হতেও দীর্ঘ সময় লাগে, ততদিনে প্রযুক্তি আরও এগিয়ে যায়।
5. **Time-to-market** চাপ (**commercial pressure**): কোম্পানিগুলো প্রতিযোগিতামূলক বাজারে দ্রুত পণ্য বাজারে আনার জন্য চাপে থাকে — এই বাণিজ্যিক তাড়াহুড়ো প্রায়ই পর্যাপ্ত testing ও safety verification-এর সময় কমিয়ে দেয় (Week 9-এ "insufficient testing" system failure-এর একটা প্রধান কারণ হিসেবে উল্লেখ ছিল)।
6. বাস্তব উদাহরণ — **Therac-25** ও **Patriot Missile**: এই দুইটা কেসই প্রমাণ করে যে, তখনকার সময়ে প্রযুক্তি (full computer control, real-time calculation) যতটা দ্রুত এগিয়েছিল, safety verification প্রক্রিয়া (hardware interlock বাদ দেওয়া, insufficient arithmetic precision

testing) ততটা এগোয়নি — এই ফাঁকই দুর্ঘটনার মূল কারণ হয়ে দাঁড়িয়েছিল।

---

তবে কিছুটা আশাবাদী দিকও আছে (বিপরীত/ভারসাম্যমূলক যুক্তি):

7. **Safety practice** ক্রমান্বয়ে উন্নত হচ্ছে (যেমন **Software Patent examination**-এর মতো): ঠিক যেমন patent office ধীরে ধীরে prior-art খোঁজার প্রক্রিয়া উন্নত করছে, তেমনি software engineering-এও automated testing tool, static analysis, formal verification-এর মতো পদ্ধতি ক্রমাগত উন্নত হচ্ছে, যা কিছুটা হলেও ফাঁক কমাতে সাহায্য করছে।
  8. অতীতের দুর্ঘটনা থেকে শেখা (**learning from failures**): Therac-25-এর মতো দুর্ঘটনার পর hardware interlock ফিরিয়ে আনা হয়েছিল — এটা দেখায় যে industry অতীতের ব্যর্থতা থেকে শিখে ভবিষ্যতের জন্য standard ও practice উন্নত করে, যদিও এটা reactive (প্রতিক্রিয়াশীল), proactive (প্রতিরোধমূলক) না।
  9. অন্য প্রযুক্তির ইতিহাস থেকে আশার আলো (**Week 9-এর "Risk is not new" concept**-এর সাথে সম্পর্কিত): গাড়ি, বিমান, বাষ্প ইঞ্জিনের ক্ষেত্রেও প্রথম দিকে নিরাপত্তা প্রযুক্তির পিছনে ছিল, কিন্তু সময়ের সাথে সাথে (দশকের পর দশক ধরে) সেগুলো অনেক নিরাপদ হয়ে উঠেছে — এটা প্রমাণ করে যে ফাঁক পুরোপুরি বন্ধ না হলেও, ধীরে ধীরে কমে আসতে পারে।
- 

সামগ্রিক সিদ্ধান্ত:

10. ভারসাম্যপূর্ণ উপসংহার: বাস্তবে software safety-র অগ্রগতি প্রযুক্তির গতির সাথে সম্পূর্ণভাবে তাল মিলিয়ে চলতে পারে না, বিশেষ করে নতুন প্রযুক্তি বাজারে আসার প্রাথমিক পর্যায়ে (যেমন এখনকার AI/generative AI প্রযুক্তির ক্ষেত্রেও একই সমস্যা দেখা যাচ্ছে — Week 10-এর AI Ethics টপিকের সাথেও এটা সরাসরি সম্পর্কিত, যেখানে bias/safety risk এখনো পুরোপুরি সমাধান হয়নি)। তবে এই ফাঁক স্থায়ী না — সময়ের সাথে সাথে, better testing tool, regulation, এবং অতীতের ব্যর্থতা থেকে শেখার মাধ্যমে এই ফাঁক ধীরে ধীরে কমে আসে, যদিও প্রযুক্তি যেহেতু থেমে থাকে না, তাই এই "ধরার চেষ্টা (catching up)" একটা চলমান, কখনো শেষ না হওয়া প্রক্রিয়া।